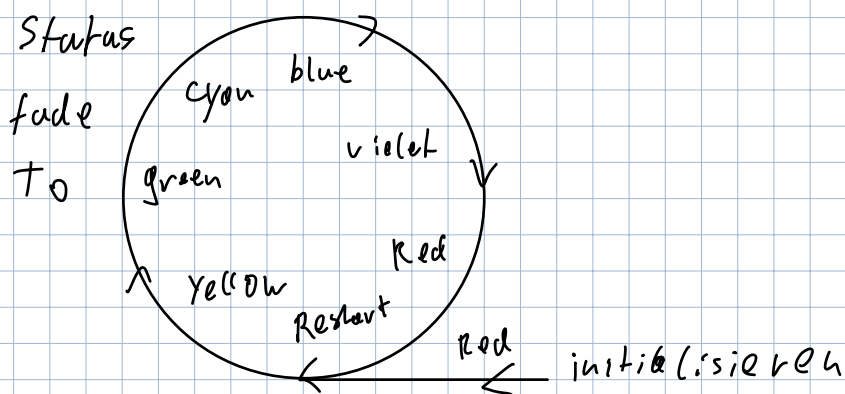


Lösungsidee:

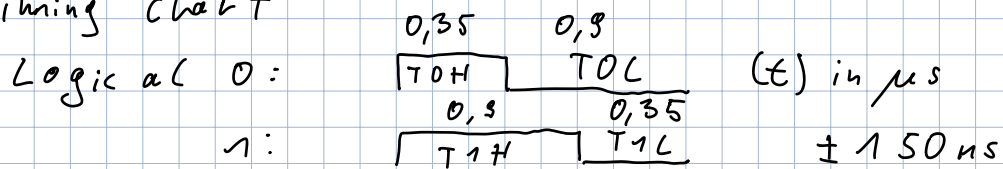
- Jeder Schritt wird als Status gespeichert
- immer wenn Schwellwert erreicht wird, kommt nächster Status
- Bei initialisierung wird rot auf max, Status auf fadeToYellow gesetzt
- Status als enum / letzter Status setzt Kette wieder auf Status fadeToYellow



- über den Status wird die zu verändernde LED (Adresse) gesucht.
- $\text{Status} \% 2 == 0 \rightarrow \text{inkrement}$
else dekrement bis Schwellwert erreicht wird

On Board LED mit WS 2812 Protokoll:
Datenpaket besteht aus 24 Bits (8G, 8R, 8B),

Timing chart

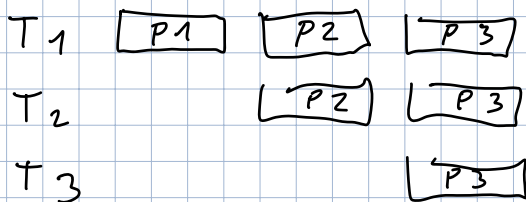


RET Code $[> 50 (> 280)]$

Zeit für 1 Bit: $1,25 \mu\text{s}$

Jeder Teilnehmer im Bus nimmt das erste
24 Bit Datenpaket und schickt die Restlichen
weiter.

Datenpakete der Teilnehmer



Das Protokoll wird beim DevKit über den
RMT-Treiber realisiert. Dieser hat einen
Puls-Generator kombiniert mit einem
Modulator. Es stehen 2 RMT-Kanäle
zur Verfügung.

Die API-Funktionen finden sich im `led_strip.h`
Header. Der GPIO wird zum RMT-Modul
"gemultiplext" mit `led_strip_init`.
Rückgabewert ist ein ptr auf ein `led_strip_t`

Anschließend können über die Funktionen `clear`, `setpixel`, `refresh` und `del` die einzelnen Aktionen ausgeführt werden.